

Month 3 – Backend + Databases: Detailed Exercises (Weeks 9-12, Days 57-84)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 12 Oct 2026 – 8 Nov 2026 (Days 57-84)

 **Your slots (IST):** Mon-Fri **7:30-8:30pm** & **9:00-10:00pm** · Sat-Sun **9:00am-12:00pm**, **3:00-5:30pm**, **6:00-8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

Roadmap: [ROADMAP.md](#) · **Theory:** [05-nodejs-express-complete.md](#), [06-databases-complete.md](#), [07-auth-devops-complete.md](#) · **Index:** [week-09-12-exercises.md](#)

WEEK 9 – Node.js + Express (Days 57-63)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Exercise W9-1: Express Server with JSON Middleware

Problem statement

Create Express server on port 4000 with `express.json()`, request logger middleware, health route `GET /health`.

Learning objective

Middleware pipeline order; basic HTTP server in Node.

Prerequisites

- Theory: [05-nodejs-express-complete.md §1-5](#)

Step-by-step approach

1. `npm init -y`; install `express`, `@types/express`, `tsx`, `typescript`.
2. `src/app.ts`: create app, use json parser, custom logger `(req, res, next)`.
3. Health returns `{ status: 'ok', timestamp }`.
4. Export app for testing; `listen` in `server.ts`.
5. Test with `curl`.

Hints

1. Middleware order: logger before routes, error handler last.
2. `express.json()` required before reading `req.body`.
3. Use `PORT` env var with fallback.

Exact Answer

- `GET /health` returns `{ status: 'ok', timestamp }` on port 4000
- Middleware order: logger → json parser → routes → error handler

Complete Solution

```
import express from "express";

const app = express();
app.use((req, _res, next) => { console.log(`${req.method} ${req.path}`); next(); });
app.use(express.json());

app.get("/health", (_req, res) => {
  res.json({ status: "ok", timestamp: new Date().toISOString() });
});

export default app;
// server.ts: app.listen(process.env.PORT || 4000);
```

Expected output / acceptance criteria

- `curl localhost:4000/health` → 200 JSON

Where to practice

- Express: <https://expressjs.com/en/starter/hello-world.html>
- Local: `mern-mastery/month-03/week-09/`

Common mistakes

- Parsing body before json middleware.
- No error handler — crashes on throw.

80 LPA Interview Twist

Amazon LP interviews use STAR with 'I' statements and dig 3 levels deep on YOUR specific contribution — team answers score poorly.

Interview connection

Foundation for all backend rounds — middleware pattern matches Next.js route handlers.

Time estimate: 45 minutes

Exercise W9-2: In-Memory Users CRUD

Problem statement

REST API `/api/users` — GET all, GET :id, POST, PUT, DELETE. In-memory array store.

Step-by-step approach

1. User model: `{ id, name, email, createdAt }`.
2. Router module `routes/users.routes.ts`.

3. Controller functions delegate to service layer stub.
4. Return proper status: 201 create, 404 missing, 204 delete optional.
5. Validate email format minimally.

Exact Answer

- Full REST CRUD on `/api/users` with proper status codes (201, 404, 204)
- In-memory store with `{ id, name, email, createdAt }`

Complete Solution

```
import { Router } from "express";
import { randomUUID } from "crypto";

const router = Router();
const users: { id: string; name: string; email: string; createdAt: string }[] = []

router.get("/", (_req, res) => res.json(users));
router.get("/:id", (req, res) => {
  const u = users.find(x => x.id === req.params.id);
  if (!u) return res.status(404).json({ error: "Not found" });
  res.json(u);
});
router.post("/", (req, res) => {
  const { name, email } = req.body;
  if (!email?.includes("@")) return res.status(400).json({ error: "Invalid email" });
  const user = { id: randomUUID(), name, email, createdAt: new Date().toISOString() };
  users.push(user);
  res.status(201).json(user);
});
router.put("/:id", (req, res) => { /* update logic */ });
router.delete("/:id", (req, res) => { /* delete logic */ });
export default router;
```

Expected output / acceptance criteria

- Full CRUD via Postman/curl documented in README.

Where to practice

- REST tutorial: <https://restfulapi.net/>

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 90 minutes

Exercise W9-3: Zod Validation Middleware

Problem statement

Reusable `validate(schema)` middleware applying Zod to `req.body`.

Step-by-step approach

1. Return middleware calling `schema.safeParse(req.body)`.
2. On failure → 400 `{ errors: issues }`.
3. On success → replace body with parsed data; `next()`.
4. Apply to POST/PUT user routes.

Hints

1. `safeParse` avoids try/catch flow.
2. Typed `req.body` needs augmentation or generic handler pattern.

Exact Answer

- Invalid body → 400 with `{ errors: ZodIssue[] }`
- Valid body replaced with parsed/transformed data

Complete Solution

```
import { ZodSchema } from "zod";
import { Request, Response, NextFunction } from "express";

export const validate = (schema: ZodSchema) => (req: Request, res: Response, next: NextFunction) => {
  const result = schema.safeParse(req.body);
  if (!result.success) return res.status(400).json({ errors: result.error.issues });
  req.body = result.data;
  next();
};
```

Where to practice

- Zod: <https://zod.dev/>

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 60 minutes

Exercise W9-4: Global Error Handler

Problem statement

Centralized error middleware mapping `AppError` to status codes; hide stack in production.

Step-by-step approach

1. Custom `AppError` extends `Error` with `statusCode`.
2. `(err, req, res, next)` handler — log err, send JSON message.
3. Async route wrapper catches rejected promises.
4. Unknown errors → 500 generic message.

Exact Answer

- `AppError` maps to specific status codes
- Unknown errors → 500 with generic message (no stack in production)

Complete Solution

```
export class AppError extends Error {
  constructor(public statusCode: number, message: string) { super(message); }
}

export const errorHandler = (err: Error, _req: Request, res: Response, _next: Next) => {
  if (err instanceof AppError) return res.status(err.statusCode).json({ error: err.message });
  console.error(err);
  res.status(500).json({ error: process.env.NODE_ENV === "production" ? "Internal Server Error" : err.message });
};

export const asyncHandler = (fn: Function) => (req: Request, res: Response, next: Next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};
```

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 45 minutes

Exercise W9-5: Layered Architecture Refactor

Problem statement

Refactor to `routes → controllers → services → repositories` (in-memory repo).

Step-by-step approach

1. Move business logic from controllers to services.
2. Repository interface for swap to DB later.
3. Controllers only HTTP concerns.
4. Diagram layers in README.

Exact Answer

- Clear separation: routes → controllers → services → repositories
- No business logic in route handlers; repository swappable for DB

Complete Solution

```
// repositories/user.repository.ts
export interface UserRepository {
  findAll(): Promise<User[]>;
  findById(id: string): Promise<User | null>;
  create(data: CreateUserDto): Promise<User>;
}
export class InMemoryUserRepository implements UserRepository { /* ... */ }

// services/user.service.ts
export class UserService {
  constructor(private repo: UserRepository) {}
  async getUser(id: string) {
    const user = await this.repo.findById(id);
    if (!user) throw new AppError(404, "User not found");
    return user;
  }
}
```

Prerequisites

- Theory: [05-nodejs-express-complete.md §6](#)

Expected output / acceptance criteria

- No DB logic in routes; each layer unit-testable in isolation.

Interview connection

"How do you structure Express apps?" — layered vs MVC vs clean architecture.

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 3 hours (Saturday)

Weekend W9: Auth API Skeleton

Problem statement

Register/login routes without DB — validate body, return mock JWT-shaped token.

Step-by-step approach

1. POST `/auth/register` — hash password with bcrypt preview.
2. POST `/auth/login` — compare hash.
3. Return `{ accessToken: 'mock...' }` for now.
4. Prepare for Week 10 real JWT.

Exact Answer

- All acceptance criteria met per step-by-step approach
- Code committed to repo with README documentation

Complete Solution

```
// Follow the 8-step build approach documented above
// Commit each step separately with meaningful messages
// Document architecture decisions in README
```

80 LPA Interview Twist

Meta/Google security rounds ask about OWASP top 10, token rotation on reuse detection, and zero-trust architecture — auth skeleton must scale to microservices.

Time estimate: 4 hours

WEEK 10 – Authentication (Days 64-70)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Exercise W10-1: bcrypt Hash & Verify

Problem statement

Utility functions `hashPassword(plain)` and `verifyPassword(plain, hash)` with cost factor 10-12.

Learning objective

Never store plaintext passwords; async bcrypt API.

Prerequisites

- Theory: [07-auth-devops-complete.md §4](#)

Step-by-step approach

1. `npm i bcrypt` or `bcryptjs`.
2. `await bcrypt.hash(plain, 12)`.
3. `await bcrypt.compare(plain, hash)`.
4. Demo script hashes same password twice — different salts.

Hints

1. bcrypt includes salt in hash string.
2. Use async APIs — sync blocks event loop.

Exact Answer

- Same password hashed twice produces different hashes (unique salts)
- `bcrypt.compare(plain, hash)` returns true for correct password

Complete Solution

```
import bcrypt from "bcryptjs";

export async function hashPassword(plain: string): Promise<string> {
  return bcrypt.hash(plain, 12);
}

export async function verifyPassword(plain: string, hash: string): Promise<boolean> {
  return bcrypt.compare(plain, hash);
}
```

Where to practice

- Local: `month-03/week-10/auth/hash.ts`

80 LPA Interview Twist

Meta/Google security rounds ask about OWASP top 10, token rotation on reuse detection, and zero-trust architecture — auth skeleton must scale to microservices.

Time estimate: 30 minutes

Exercise W10-2: JWT Access Token Generation

Problem statement

Sign JWT with payload `{ userId, role }`, expiry 15m, secret from env.

Step-by-step approach

1. `npm i jsonwebtoken @types/jsonwebtoken`.
2. `jwt.sign(payload, process.env.JWT_SECRET!, { expiresIn: '15m' })`.
3. Verify with `jwt.verify` in test script.
4. Never commit secret — `.env.example` only.

Exact Answer

- Access token expires in 15m; payload `{ userId, role }`
- Verify with same secret; expired tokens rejected

Complete Solution

```
import jwt from "jsonwebtoken";

export function signAccessToken(payload: { userId: string; role: string }) {
  return jwt.sign(payload, process.env.JWT_SECRET!, { expiresIn: "15m" });
}

export function verifyAccessToken(token: string) {
  return jwt.verify(token, process.env.JWT_SECRET!) as { userId: string; role: string };
}
```

Where to practice

- jwt.io debugger: <https://jwt.io/>

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and

production tradeoffs — not just happy-path implementation.

Time estimate: 45 minutes

Exercise W10-3: Refresh Token in httpOnly Cookie

Problem statement

On login, set refresh token cookie; access token in JSON body.

Step-by-step approach

1. Issue long-lived refresh token (7d) stored signed or in DB later.
2. `res.cookie('refreshToken', token, { httpOnly: true, secure: prod, sameSite: 'strict' })`.
3. POST `/auth/refresh` reads cookie, validates, issues new access token.

Hints

1. httpOnly prevents XSS theft from JS.
2. Rotation: invalidate old refresh on use (Weekend project).

Exact Answer

- Login returns access token in JSON + refresh token in httpOnly cookie
- POST `/auth/refresh` reads cookie, validates, issues new access token

Complete Solution

```
router.post("/login", async (req, res) => {
  // verify credentials...
  const accessToken = signAccessToken({ userId: user.id, role: user.role });
  const refreshToken = signRefreshToken({ userId: user.id });
  res.cookie("refreshToken", refreshToken, {
    httpOnly: true, secure: process.env.NODE_ENV === "production", sameSite: "strict"
  });
  res.json({ accessToken });
});
```

80 LPA Interview Twist

Meta/Google security rounds ask about OWASP top 10, token rotation on reuse detection, and zero-trust architecture — auth skeleton must scale to microservices.

Time estimate: 90 minutes

Exercise W10-4: Auth Middleware

Problem statement

`authenticate` middleware reads `Authorization: Bearer <token>` , attaches `req.user` .

Step-by-step approach

1. Parse header; verify JWT.
2. On failure 401; on success `next()` .
3. Protect sample `/api/profile` route.

Exact Answer

- Missing/invalid Bearer token → 401
- Valid token attaches `req.user` and calls `next()`

Complete Solution

```
export function authenticate(req: Request, res: Response, next: NextFunction) {
  const header = req.headers.authorization;
  if (!header?.startsWith("Bearer ")) return res.status(401).json({ error: "Unauth"
  try {
    req.user = verifyAccessToken(header.slice(7));
    next();
  } catch {
    res.status(401).json({ error: "Invalid token" });
  }
}
```

80 LPA Interview Twist

Meta/Google security rounds ask about OWASP top 10, token rotation on reuse detection, and zero-trust architecture — auth skeleton must scale to microservices.

Time estimate: 45 minutes

Exercise W10-5: RBAC Admin vs User

Problem statement

`authorize('admin')` middleware checks role after authenticate.

Step-by-step approach

1. Role in JWT payload.

2. Factory `authorize(...roles)` returns middleware.
3. Admin-only DELETE route; user gets 403.

Exact Answer

- User role token → 403 on admin-only route
- Admin role token → 200 on admin route

Complete Solution

```
export const authorize = (...roles: string[]) => (req: Request, res: Response, next: NextFunction) => {
  if (!req.user || !roles.includes(req.user.role)) return res.status(403).json({ error: 'Forbidden' });
  next();
};
// router.delete("/users/:id", authenticate, authorize("admin"), deleteUser);
```

Expected output / acceptance criteria

- User token → 403 on admin route; admin token → 200.

Interview connection

AuthN vs AuthZ distinction — draw token flow diagram in interviews.

80 LPA Interview Twist

Meta/Google security rounds ask about OWASP top 10, token rotation on reuse detection, and zero-trust architecture — auth skeleton must scale to microservices.

Time estimate: 60 minutes

Weekend W10: Complete Auth Microservice

Problem statement

Full refresh rotation, logout invalidates refresh, env-based secrets.

Step-by-step approach

1. Store refresh token hash in memory/Redis stub.
2. Rotate on refresh; detect reuse → revoke all sessions.
3. Logout clears cookie + server-side token.
4. Postman collection for all flows.

Exact Answer

- All acceptance criteria met per step-by-step approach

- Code committed to repo with README documentation

Complete Solution

```
// Follow the 8-step build approach documented above  
// Commit each step separately with meaningful messages  
// Document architecture decisions in README
```

80 LPA Interview Twist

Meta/Google security rounds ask about OWASP top 10, token rotation on reuse detection, and zero-trust architecture — auth skeleton must scale to microservices.

Time estimate: 6 hours

WEEK 11 – MongoDB (Days 71-77)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Exercise W11-1: Mongoose User Schema

Problem statement

User schema: email unique, password hash, role enum, timestamps.

Step-by-step approach

1. MongoDB Atlas cluster — whitelist IP, connection string in `.env`.
2. `mongoose.connect` on startup.
3. Schema with validation: email lowercase trim.
4. Index on email unique.

Exact Answer

- User schema with unique email index, role enum, timestamps
- Password stored as hash, never returned in queries

Complete Solution

```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  passwordHash: { type: String, required: true },
  role: { type: String, enum: ["user", "admin"], default: "user" },
}, { timestamps: true });

userSchema.index({ email: 1 }, { unique: true });
export const User = mongoose.model("User", userSchema);
```

Where to practice

- MongoDB Atlas: <https://www.mongodb.com/cloud/atlas/register>
- Mongoose: <https://mongoosejs.com/docs/guide.html>

80 LPA Interview Twist

Amazon asks when to embed vs reference, shard key selection, and eventual consistency tradeoffs for global deployments.

Time estimate: 60 minutes

Exercise W11-2: CRUD with Atlas

Problem statement

Replace in-memory user repo with Mongoose methods.

Step-by-step approach

1. `User.create`, `findById`, `findOneAndUpdate`, `deleteOne`.
2. Handle duplicate email E11000 → 409 Conflict.
3. Never return password field — `.select('-password')`.

Exact Answer

- Follow acceptance criteria in Expected output section
- Implementation matches step-by-step approach requirements

Complete Solution

```
const result = await User.aggregate([
  { $group: { _id: "$role", count: { $sum: 1 } } },
  { $sort: { count: -1 } },
  { $project: { role: "$_id", count: 1, _id: 0 } },
]);
// [{ role: "user", count: 150 }, { role: "admin", count: 5 }]
```

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 90 minutes

Exercise W11-3: Pagination

Problem statement

`GET /api/users?page=1&limit=10` returns `{ data, total, page, pages }`.

Step-by-step approach

1. `skip = (page-1)*limit`; `limit` cap at 100.

2. `Promise.all([find().skip().limit(), countDocuments()])`.
3. Validate query params with Zod.

Exact Answer

- `GET /api/users?page=1&limit=10` → `{ data, total, page, pages }`
- Limit capped at 100

Complete Solution

```
router.get("/", async (req, res) => {
  const page = Math.max(1, parseInt(req.query.page as string) || 1);
  const limit = Math.min(100, parseInt(req.query.limit as string) || 10);
  const skip = (page - 1) * limit;
  const [data, total] = await Promise.all([
    User.find().select("-passwordHash").skip(skip).limit(limit),
    User.countDocuments(),
  ]);
  res.json({ data, total, page, pages: Math.ceil(total / limit) });
});
```

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 60 minutes

Exercise W11-4: Aggregation Pipeline

Problem statement

Count users by role; optional: posts per month if blog data exists.

Step-by-step approach

1. `$group: { _id: '$role', count: { $sum: 1 } }`.
2. `$sort`, `$project` for clean output.
3. Run in Mongo Compass or script.

Exact Answer

- Follow acceptance criteria in Expected output section
- Implementation matches step-by-step approach requirements

Complete Solution

```
// prisma/schema.prisma + migrate
npx prisma migrate dev --name init
npx prisma generate
const users = await prisma.user.findMany({ include: { orders: true } });
```

Where to practice

- Aggregation: <https://www.mongodb.com/docs/manual/aggregation/>

80 LPA Interview Twist

Amazon asks when to embed vs reference, shard key selection, and eventual consistency tradeoffs for global deployments.

Time estimate: 75 minutes

Exercise W11-5: Index & Explain Plan

Problem statement

Add index on email; compare `explain('executionStats')` with/without index.

Step-by-step approach

1. Query by email before index — COLLSCAN.
2. Create index; IXSCAN expected.
3. Document in README screenshot or stats.

Exact Answer

- Follow acceptance criteria in Expected output section
- Implementation matches step-by-step approach requirements

Complete Solution

```
SELECT u.id, u.name, COALESCE(SUM(o.total), 0) AS order_total
FROM users u LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id, u.name;
```

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 45 minutes

Weekend W11: Blog API MongoDB

Problem statement

Posts, comments, tags — nested comments optional; auth author on POST.

Step-by-step approach

1. Schemas: Post, Comment with refs.
2. Populate author on GET.
3. Tag filter query.
4. Seed script with sample data.

Exact Answer

- All acceptance criteria met per step-by-step approach
- Code committed to repo with README documentation

Complete Solution

```
// Follow the 8-step build approach documented above
// Commit each step separately with meaningful messages
// Document architecture decisions in README
```

80 LPA Interview Twist

Meta/Google security rounds ask about OWASP top 10, token rotation on reuse detection, and zero-trust architecture — auth skeleton must scale to microservices.

Time estimate: 6 hours

WEEK 12 – PostgreSQL + MySQL + Redis (Days 78–84)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Exercise W12-1: PostgreSQL Schema users + orders

Problem statement

SQL schema with FK: users 1—N orders; orders have total, status.

Step-by-step approach

1. Neon or Supabase free DB.
2. Write SQL migration or Prisma schema.
3. FK `orders.user_id REFERENCES users(id)`.

Exact Answer

- Follow acceptance criteria in Expected output section
- Implementation matches step-by-step approach requirements

Complete Solution

```
# Vercel (frontend): vercel deploy --prod
# Render/Railway (API): connect repo, set env vars, deploy
# Verify: curl https://api.example.com/health
# Update README with live demo URLs
```

Where to practice

- Neon: <https://neon.tech/>
- Supabase: <https://supabase.com/>
- PostgreSQL tutorial: <https://www.postgresqltutorial.com/>

80 LPA Interview Twist

Google Cloud SQL interviews cover connection pooling (PgBouncer), read replicas, and EXPLAIN ANALYZE for query optimization at scale.

Time estimate: 60 minutes

Exercise W12-2: Prisma Setup + Migrations

Problem statement

Initialize Prisma; migrate; generate client.

Step-by-step approach

1. `npx prisma init`.
2. Define models User, Order.
3. `prisma migrate dev --name init`.
4. Seed script optional.

Exact Answer

- `npx prisma migrate dev` creates migration SQL; client generated in `node_modules/.prisma`
- Models User + Order with FK relation compile and migrate cleanly

Complete Solution

```
npx prisma init
# define models in schema.prisma, then:
npx prisma migrate dev --name init
npx prisma generate
npx prisma db seed # optional
```

Where to practice

- Prisma: <https://www.prisma.io/docs/getting-started>

80 LPA Interview Twist

Amazon LP interviews use STAR with 'I' statements and dig 3 levels deep on YOUR specific contribution — team answers score poorly.

Time estimate: 75 minutes

Exercise W12-3: JOIN Query – Order Totals per User

Problem statement

Query users with sum of order totals using Prisma or raw SQL.

Step-by-step approach

1. Raw: `SELECT u.id, SUM(o.total) FROM users u LEFT JOIN orders o ... GROUP BY u.id`.

2. Prisma: `groupBy` or `_sum` aggregate.
3. Expose via API endpoint.

Exact Answer

- All acceptance criteria from Expected output section met
- Implementation follows step-by-step approach; tested and documented in README

Complete Solution

```
// Follow the step-by-step approach documented above for this exercise.  
// Key deliverables: working code, tests, README documentation.  
// Verify with: npm test && curl/httpie manual testing
```

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 60 minutes

Exercise W12-4: Redis Cache on GET /users/:id

Problem statement

Cache user by id 60s TTL; cache-aside pattern.

Step-by-step approach

1. Redis via Upstash or local Docker.
2. Check cache key `user:${id}` ; on miss query DB, set cache.
3. Invalidate on user update/delete.

Exact Answer

- Cache hit → return from Redis without DB query
- Cache miss → query DB, set cache with 60s TTL
- Update/delete invalidates cache key

Complete Solution

```
async function getUserById(id: string) {  
  const cached = await redis.get(`user:${id}`);  
  if (cached) return JSON.parse(cached);  
  const user = await User.findById(id).select("-passwordHash");  
  if (user) await redis.setex(`user:${id}`, 60, JSON.stringify(user));  
  return user;  
}
```

Where to practice

- Upstash: <https://upstash.com/>

80 LPA Interview Twist

Netflix/Meta use Redis for rate limiting, pub/sub, and session store — expect distributed Redis cluster and failover discussion.

Time estimate: 90 minutes

Exercise W12-5: Rate Limiter 100 req/min per IP

Problem statement

Middleware using Redis INCR with expiry window.

Step-by-step approach

1. Key `rate:${ip}:${minuteWindow}`.
2. Increment; if > 100 return 429 with Retry-After.
3. Apply globally or per-route.

Exact Answer

- >100 requests/minute per IP → 429 with `Retry-After` header
- Redis INCR with window expiry

Complete Solution

```
export async function rateLimiter(req: Request, res: Response, next: NextFunction) {
  const ip = req.ip;
  const window = Math.floor(Date.now() / 60000);
  const key = `rate:${ip}:${window}`;
  const count = await redis.incr(key);
  if (count === 1) await redis.expire(key, 60);
  if (count > 100) return res.status(429).set("Retry-After", "60").json({ error: "Too many requests" });
  next();
}
```

Interview connection

Rate limiter LLD in Month 5 connects directly.

80 LPA Interview Twist

Netflix/Meta use Redis for rate limiting, pub/sub, and session store — expect distributed Redis cluster and failover discussion.

Time estimate: 75 minutes

Exercise W12-6: MySQL Schema + mysql2 Connection

Problem statement

Create the same users + orders schema in **MySQL 8.0** (InnoDB, utf8mb4). Connect via `mysql2/promise` connection pool.

Step-by-step approach

1. Run MySQL locally (Docker: `mysql:8.0`) or use PlanetScale/Railway free tier.
2. Write SQL with `ENGINE=InnoDB`, `utf8mb4`, FK on `orders.user_id`.
3. Create pool with `mysql2/promise`; parameterized `execute()` for SELECT/INSERT.
4. Verify with a seed user + order.

Exact Answer

- Tables `users` and `orders` exist with FK constraint
- Connection pool returns rows via parameterized query (no string concatenation)
- Charset is `utf8mb4_unicode_ci`

Complete Solution

```
import mysql from "mysql2/promise";

const pool = mysql.createPool({
  host: process.env.MYSQL_HOST ?? "localhost",
  user: process.env.MYSQL_USER ?? "root",
  password: process.env.MYSQL_PASSWORD,
  database: process.env.MYSQL_DATABASE ?? "blog_db",
  waitForConnections: true,
  connectionLimit: 10,
});

export async function createUser(name: string, email: string) {
  const [result] = await pool.execute<mysql.ResultSetHeader>(
    "INSERT INTO users (name, email) VALUES (?, ?)",
    [name, email]
  );
  return result.insertId;
}

export async function getUserWithOrders(userId: number) {
  const [rows] = await pool.execute(
    `SELECT u.id, u.name, u.email, o.id AS order_id, o.total, o.status
    FROM users u LEFT JOIN orders o ON o.user_id = u.id WHERE u.id = ?`,
    [userId]
  );
  return rows;
}
```

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(255) NOT NULL,
  created_at DATETIME(6) DEFAULT CURRENT_TIMESTAMP(6),
  UNIQUE KEY uk_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE orders (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  total DECIMAL(10,2) NOT NULL,
  status ENUM('pending', 'paid', 'shipped') DEFAULT 'pending',
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  INDEX idx_user_id (user_id)
) ENGINE=InnoDB;
```

Where to practice

- MySQL Docker: https://hub.docker.com/_/mysql
- PlanetScale: <https://planetscale.com/>
- mysql2: <https://github.com/sidorares/node-mysql2>

80 LPA Interview Twist

Flipkart/Uber interviews ask PostgreSQL vs MySQL tradeoffs, InnoDB gap locks, and when Vitess/PlanetScale sharding beats single-instance MySQL.

Time estimate: 75 minutes

Exercise W12-7: Prisma with MySQL Provider

Problem statement

Rebuild W12-1 schema using Prisma with `provider = "mysql"`. Run migrations; compare with PostgreSQL version.

Step-by-step approach

1. `DATABASE_URL="mysql://user:pass@localhost:3306/blog_db" npx prisma init`
2. Set `provider = "mysql"` in `schema.prisma`.
3. Define User + Order models with `@db.VarChar`, `@db.Decimal`.
4. `npx prisma migrate dev --name init`
5. Write 3-bullet comparison: PG vs MySQL for this schema.

Exact Answer

- Migration applies cleanly on MySQL 8.0
- Prisma Client generates types; CRUD works
- Comparison doc covers isolation defaults, JSONB vs JSON, and managed hosting options

Complete Solution

```
datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
}

model User {
  id        Int      @id @default(autoincrement())
  name      String    @db.VarChar(100)
  email     String    @unique @db.VarChar(255)
  orders    Order[]
  createdAt DateTime @default(now()) @map("created_at")
  @@map("users")
}

model Order {
  id        Int      @id @default(autoincrement())
  userId    Int      @map("user_id")
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  total     Decimal  @db.Decimal(10, 2)
  status    String   @default("pending") @db.VarChar(20)
  @@index([userId])
  @@map("orders")
}
```

PostgreSQL vs MySQL (this project)

- PostgreSQL: JSONB, Read Committed default, Neon/Supabase – pick for analytics
- MySQL: Repeatable Read default, PlanetScale/RDS – pick for high-read OLTP
- Prisma: same API; swap provider + DATABASE_URL only

Where to practice

- Prisma MySQL guide: <https://www.prisma.io/docs/orm/overview/databases/mysql>

80 LPA Interview Twist

Amazon RDS interviews cover read replicas, connection pool sizing, and EXPLAIN for slow queries on both PG and MySQL.

Time estimate: 60 minutes

Exercise W12-8: EXPLAIN ANALYZE – PostgreSQL vs MySQL

Problem statement

Run the same JOIN query on PostgreSQL and MySQL. Compare EXPLAIN output; add index if full

table scan.

Step-by-step approach

1. Query: top 10 users by total order value (JOIN + GROUP BY + ORDER BY).
2. PostgreSQL: `EXPLAIN ANALYZE SELECT ...`
3. MySQL: `EXPLAIN ANALYZE SELECT ...` (8.0.18+)
4. Add composite index on `(user_id, status)` if needed.
5. Document before/after row counts in README.

Exact Answer

- Both databases show index usage after optimization (no `ALL` / seq scan on large tables)
- README table comparing EXPLAIN columns (`Seq Scan` vs `type: ALL`)

Complete Solution

```
-- Shared query pattern
SELECT u.id, u.name, COALESCE(SUM(o.total), 0) AS total_spent
FROM users u
LEFT JOIN orders o ON o.user_id = u.id AND o.status = 'paid'
GROUP BY u.id, u.name
ORDER BY total_spent DESC
LIMIT 10;

-- PostgreSQL index
CREATE INDEX idx_orders_user_status ON orders(user_id, status);

-- MySQL index
CREATE INDEX idx_orders_user_status ON orders(user_id, status);
```

80 LPA Interview Twist

Meta/Google expect you to explain leftmost prefix rule, covering indexes, and when denormalization beats JOINS at scale.

Time estimate: 45 minutes

Weekend W12: Blog API PostgreSQL Rebuild

Problem statement

Rebuild Week 11 blog with Prisma/Postgres or **MySQL**; README comparing Mongo vs PostgreSQL vs MySQL tradeoffs.

Step-by-step approach

1. Relational models: posts, comments, tags junction table.
2. Transactions for order-like operations demo.
3. Document when to pick each DB.

Exact Answer

- All acceptance criteria met per step-by-step approach
- Code committed to repo with README documentation

Complete Solution

```
// Follow the 8-step build approach documented above
// Commit each step separately with meaningful messages
// Document architecture decisions in README
```

Expected output / acceptance criteria

- Comparison doc: schema flexibility, joins, scaling, ACID, PG vs MySQL vs Mongo.

80 LPA Interview Twist

Google Cloud SQL interviews cover connection pooling (PgBouncer), read replicas, and EXPLAIN ANALYZE for query optimization at scale.

Time estimate: 6 hours

Month 3 LeetCode Schedule

Week	Focus Problems
W9	LC 136, 141, 155, 160, 167
W10	LC 169, 171, 190, 191, 202
W11	LC 203, 206, 217, 219, 226
W12	LC 230, 235, 238, 242, 252

Use Week 1 detailed template for each: approach, hints, acceptance criteria.

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

---## Month 3 Completion Checklist

Milestone	Done
Layered Express API	☐
JWT + refresh + RBAC	☐
MongoDB blog API	☐
PostgreSQL + Prisma rebuild	☐
MySQL schema + mysql2 or Prisma mysql	☐
Redis cache + rate limit	☐
100+ LeetCode total	☐

Total estimated time: ~80–96 hours